Algorithms for Data Science
Practice Exam
(with Solutions)

See the previous exercise sheets for more examples of tasks that can appear in the exam.
Note: in the questions asking you to "explain", include a brief explanation of your answer (a few words).

Task 1:

What are the time complexities of the following programs in Θ notation in dependence of n?

(a)

```
for x = 1 to n do
    a = a + 1
    for y = x − 10 to x + 10 do
        a = a + 1
```

Time: $\Theta(n)$

Explanation (not required):
Observe that the inner for loop repeats only 21 times, which is a constant. Thus, each iteration of the outer for loop takes $O(1)$ time.

(b)

```
for x = 1 to 10n do
    y = 10n
    while y ≥ x do
        y = y − 1
```

Time: $\Theta(n^2)$

Explanation (not required):
The running time is proportional to the total number of times we perform $y = y - 1$ (our dominating operation). For each x, the inner loop is repeated $10n - x + 1$ times. Thus, the total number of times we perform $y = y - 1$ is

$$\sum_{x=1}^{10n}(10n - x + 1) = \sum_{x=1}^{10n} x \ .$$

By the summation rule, we have

$$\sum_{x=1}^{10n} x = (10n)(10n + 1)/2$$

which in asymptotics notation can be simplified to $\Theta((10n)^2) = \Theta(n^2)$.

Task 2:

Suppose we have an algorithm X which, given the input x, proceeds as follows:

- if x is of size at most 1, it computes the output in time $O(1)$,
- if x is of size $n > 1$, it recursively calls X on four inputs each of size $\lceil n/2 \rceil$. After the four calls, it combines the results and computes the output in time $\Theta(n^2)$.

What is the time complexity of the algorithm X in Θ notation? Explain your answer.

Solution: The running time satisfies the following recurrence:

$$T(n) = 4T(\lceil n/2 \rceil) + \Theta(n^2) \ .$$

By the master theorem, we have $a = 4$, $b = 2$, $f(n) = \Theta(n^2)$.
Further, we have $c = \log_b a = \log_2 4 = 2$.
Thus, Case B of the master theorem holds as $f(n) = \Theta(n^2) = \Theta(n^c)$. Therefore, the resulting running time is $T(n) = \Theta(n^c \log n) = \Theta(n^2 \log n)$.

Task 3:

We are given a directed graph with n vertices numbered from 0 to $n-1$, and m edges given as a list of pairs (source vertex, target vertex). What algorithm can we use to sort the edges in time and memory $O(n+m)$? The edges should be sorted by their source vertices, and edges from the same vertex should be sorted by their target vertex. Explain your answer.

Solution: Using `Breadth First Search` is wrong even though its running time is $O(n+m)$! The correct solution is to use `Radix Sort` or twice `Counting Sort` as explained below: First sort the edges via `Counting Sort` with respect to the target vertices, then another time with respect to the source vertices. (Hence, after the first sort, any two edges with the same source vertex are already correctly sorted relative to each other (i.e., when ignoring other edges); the second sort ensures that any two edges with different source vertices are also correctly sorted relative to each other.) The running time of each call of `Counting Sort` is $O(n+m)$ as each time we sort m objects (edges) with values from 0 to $n-1$ (target/source vertices). The total time is thus also $O(n+m)$.

Task 4:

Write an algorithm based on dynamic programming that solves the same problem as `AlgoRec(`a`, `n`)`. Assume that n is an integer of size at least 2 and that a is an array $a[2..n]$ of length $n-1$ starting at index 2.

Hint: You don't have to understand the problem. You just have to transform the algorithm below.

```
memo = new dictionary
AlgoRec(a[2..n], p)
    if p < 2 then
        return 0
    if memo.Check(p) == False then
        memo.Add(p, max(a[p] + AlgoRec(a,p-2), AlgoRec(a,p-1)))
    return memo.Get(p)
```

Solution:

```
DP[0..n] = new array
DP[0] = 0
DP[1] = 0
for p = 2 to n do
    DP[p] = max(a[p] + DP[p-2], DP[p-1])
return DP[n]
```

Task 5:

Draw a binary search tree of height 3 containing the keys $40, 10, 30, 20$ such that it is *not* an AVL tree. Mark the vertex in your tree that violates the invariant of the AVL tree.

Solution: There are two possible solutions:

Task 6:

In each subtask, suppose that our algorithm executes the given number of dominating operations in the wort case where n is the size of the input. Give the time complexity simplified as far as possible using the Θ notation.

(a) $4n + \log n + 1$

Answer: $\Theta(n)$

(b) $3n \log n^3 + 100n$

Answer: $\Theta(n \log n)$

Explanation (not required):
Recall that, for any constant c, $\Theta(\log n^c) = \Theta(\log n)$.

(c) $10 \log^{10} n + \sqrt{n}/100 + \sqrt{2}^{\log n} \cdot \log n$

Answer: $\Theta(\sqrt{n} \log n)$

Explanation (not required):
Use the fact $\sqrt{2} = 2^{1/2}$ and the transformation rules:

$$\sqrt{2}^{\log n} = (2^{1/2})^{\log n} = 2^{\log n/2} = (2^{\log n})^{1/2} = n^{1/2} = \sqrt{n}.$$

Thus, the last of the three terms is $\sqrt{n} \log n$ which grows faster (is larger) than the other ones.

Task 7:

Give the worst case time complexity of the following algorithms in Θ notation in dependence of n:

(a)

Input: an array $a[1..n]$
$J =$ `make_heap`(a)
for $p = n$ **to** 1 **do**
    J.`removeMax`()

Time: $\Theta(n \log n)$

Explanation (not required):
The algorithm above is precisely heap sort which has this running time.

(b)

Input: *edges*: a list with $O(n^2)$ edges of a graph that has n vertices numbered from 1 to n
for $v = 1$ **to** n **do**
    for $w = 1$ **to** n **do**
        $\delta =$ `Floyd_Warshall(`*edges*`)`
        `print(`$\delta[v][w]$`)`

Time: $\Theta(n^5)$

Explanation (not required):
We run Floyd-Warshall n^2 times, each time with a running time of $\Theta(n^3 + m) = \Theta(n^3)$ (given that the number of edges $m = O(n^2)$).

(c)

Input: an array $a[1..n]$, a sorted array $b[1..n]$
for $i = 1$ **to** n **do**
    Add $a[i]$ into b such that b is still sorted

Time: $\Theta(n^2)$

Explanation (not required):
Though, for each $a[i]$, we can find the right position in $O(\log n)$ time in b, we need $\Theta(n)$ in the worst case to shift the other elements to the side to make the position free.

(d)

Input: an AVL tree T with n elements, an array $a[1..100n^2]$ with $100 \cdot n^2$ elements.
for $i = 1$ **to** $100 \cdot n \cdot n$ **do**
  T.`add`($a[i]$)

Time: $\Theta(n^2 \log n)$

Explanation (not required):
The add operation of an AVL tree with k elements takes $\Theta(\log k)$ time in the worst case. The AVL tree has, during the life time of the algorithm above, always between n and $100n^2 + n$ elements. Thus, every add operation takes between $\Theta(\log n)$ and $\Theta(\log(100n^2 + n)) = \Theta(\log(n^2)) = \Theta(\log n)$ time. Hence, every add operation takes just $\Theta(\log n)$ time. And there are $\Theta(n^2)$ of such add operations.

Task 8:

Give the worst case *extra*[1] memory complexity of the following algorithms in Θ notation in dependence of n:

(a)

Input: an array $a[1..n]$
`merge_sort`(a)
return $a[0]$

Memory: $\Theta(n)$

Explanation (not required):
The memory complexity of merge sort.

(b)

Input: an array $a[1..n]$
$x = 2 \cdot a[0]$
return `binary_search`(a, x)

Memory: $\Theta(1)$

Explanation (not required):
The extra space of space of binary search and the single extra variable x.

[1] not counting the memory for the input

(c)

Input: a list *adjacency_list* representing a weighed graph with n vertices numbered from 1 to n and $O(n)$ edges,
two vertices s and t (given as numbers from 1 to n)
dist = `dijkstra_algorithm(`*adjacency_list, s*`)`
return *dist*[*s*][*t*]

Memory: $\Theta(n)$

Explanation (not required):
The *dist* table ($=\Theta(n)$) as well as the priority queue ($=\Theta(m) = \Theta(n)$ as number m of edges is $O(n)$) inside Dijkstra's algorithm take linear space.

(d) The same algorithm as before, but this time the graph has $\Theta(n^2)$ edges.

Memory: $\Theta(n^2)$

Explanation (not required):
Now $m = \Theta(n^2)$, thus the priority queue in Dijkstra contains $\Theta(n^2)$ vertices (with duplicates) in the worst case.

Task 9:

Consider the following Python and C++ programs labeled as A, B, C, and D, operating on a list/vector a containing n integers.

Order the four programs by their actual real running time assuming that n is very large and that the programs are executed on the same computer. Briefly explain your reasoning for the ordering.

Recall that the Python operation `a.pop(i)` removes $a[i]$ from a. If you are unfamiliar with C++, it suffices for you to know that Program C does the same as Program A, and Program D does the same as Program B.

- Program A in Python

```
del a[0]
for _ in range(n // 2):
        a.pop(0)
```

- Program B in Python

```
for _ in range(n // 2):
        a.pop(len(a)-1)
```

- Program C in C++

```
a.erase(a.begin());
for (int i = 0; i < n / 2; ++i) {
        a.erase(a.begin());
}
```

- Program D in C++

```
for (int i = 0; i < n / 2; ++i) {
        a.pop_back();
}
```

Solution: The correct order of the four programs by their actual running time for large n:

D, B, C, A

Brief reasoning:

- In practice, C++ programs are faster than equivalent Python programs.
- Programs D and B are $\Theta(n)$ because they perform $\Theta(n)$ efficient removals.
- Programs A and C are $\Theta(n^2)$ due to the inefficient removal of elements from the beginning, which requires shifting of all subsequent elements.

Detailed Explanation (not required):

- Programs A and C: The first element is removed $n/2$ times. Each time, all other elements have to be moved by one position to the left which takes a lot of time. To be more precise, we have to shift at least $n/2$ elements, thus the time is $\Theta(n)$ per element removal. The total time is therefore $\Theta(n^2)$. Since an equivalent program in C++ is normally faster than in Python (most likely by some constant factor), we have that C is faster than A.
- Programs B and D: The last element is removed $n/2$ times. Each time, the removal operation essentially boils down to just ignore the last element (see discussion below). Thus, the time per element removal is $O(1)$ (amortized). The total time is therefore $\Theta(n)$. Since B and D

are equivalent programs, and B is in Python and D in C++, D is faster than B. On the other hand, since B is asymptotically faster than A and C, for large enough n it will be faster in practice, too (even though A is a C++ program).

Background (see Lecture 2): The elements of an array (or pointers to these) are stored in a consecutive block in the memory. The Python list object and the C++ vector only stores the position where this block starts and how many elements it contains. If we remove the last element, we essentially only decrease the counter of elements by one, which can be done in $O(1)$ amortized time. However, if we remove the first element, then we not only decrease the counter, but we also move the whole block (without the first element) one step to the left in the memory which takes always linear time.

Task 10:

Is it possible to sort n elements with $O(n\alpha(n))$ comparisons in the worst case (where $\alpha(n)$ is the inverse Ackermann function)? Explain.

Solution: In the worst case, we need at least $\Omega(n \log n)$ comparisons to sort n elements. Since the function $\alpha(n)$ is growing much slower than $\log n$, so is also $n\alpha(n)$ compared to $n \log n$. Thus, we conclude that $O(n\alpha(n))$ comparisons are not enough.

Task 11:

What is the most appropriate data structure for the following scenario (in terms of memory and time complexity) from the list below? Give a short explanation of your answer.

In your answer, mention the memory and time complexity of your data structure. Also argue why your data structure is more appropriate than the other ones.

Initially, there are n bacteria on a petri dish, numbered from 1 to n. The bacteria are growing and whenever two bacteria meet, they become a new single bacterium. A camera registers such events and sends the data to a computer. Propose a data structure that can quickly tell whether any given two initial bacteria (given by their ID numbers) already belong to a same new bacterium or are still isolated from each other.

1. Array indexed by keys
2. AVL tree
3. Find-union
4. Hash table
5. Heap/Priority queue
6. Sorted Linked list
7. Queue
8. Stack
9. Sorted array
10. Unsorted array

Most appropriate data structure: **Find-union**

Explanation:

Solution: We use find-union.

At the beginning, we call `Init(`n`)` to create n disjoint sets, one for each bacteria.

Whenever two bacteria i and j meet, we call `Union(`i`,`j`)` to join the sets of both bacteria together.

If someone asks whether two initial bacteria, i and j, belong to the same bacterium, we return "Yes" if `Find(`i`)` (the ID of the set containing i) is equal to `Find(`j`)`; otherwise, we return "No".

The memory usage is optimal at $\Theta(n)$, where n is the number of elements.

The running time per operation is $\Theta(\alpha(n))$, where α is the inverse Ackermann function. This is almost constant and extremely fast.

All the other data structures manage collections of individual keys instead of collections of disjoint sets that can be dynamically merged.

Task 12:

Consider the following incomplete Python function to determine whether it's possible to reach the last field from the first field given the constraints. The input is a Python list (array) *level* of length $n \geq 4$ containing only values 0 and 1, as well as a positive integer w. The goal is to reach field $n-1$ from field 0. If $level[i] = 1$, you can walk to the right, or jump to the field $i + w$. You can jump at most three times. If $level[i] = 0$, you can't move further.

Note: If you have problems with the first subtask, take a look at the second subtask to get a right idea for the algorithm.

```
1  def can_reach(level, w):
2      n = len(level)
3      # ??? [Line 3]
4      # ??? [Line 4]
5      # ??? [Line 5] (Starts with for...)
6          if level[i - 1] == 1:
7              # ??? [Line 7]
8          if i - w >= 0 and level[i - w] == 1:
9              # ??? [Line 9]
10     # ??? [Line 10] (Starts with return...)
```

(a) Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct label from the code snippets below:

- Line 3: **H**
- Line 4: **J**
- Line 5: **D**
- Line 7: **B**
- Line 9: **C**
- Line 10: **L**

Code snippets to use:

- (A) `return dp[n - 1] <= n`
- (B) `dp[i] = dp[i - 1]`
- (C) `dp[i] = min(dp[i], dp[i - w] + 1)`
- (D) `for i in range(1,n):`
- (E) `dp[i] = min(dp[i - 1], dp[i - w] + 1)`
- (F) `dp = [0] * n`
- (G) `for i in range(0,n):`
- (H) `dp = [n] * n`
- (I) `return dp[n - 1] > -1`
- (J) `dp[0] = 0`
- (K) `dp = [-1] * n`
- (L) `return dp[n - 1] < 4`
- (M) `dp[i] = dp[i - w] + 1`
- (N) `dp[i] = min(dp[i] + 1, dp[i - w])`
- (O) `dp[0] = n`
- (P) `dp[0] = -1`
- (Q) `for i in range(1,n-1):`
- (R) `dp[0] = 1`
- (S) `dp[i] = dp[i - 1] + 1`
- (T) `for i in range(0,n-1):`
- (U) `return dp[n - 1] >= 3`
- (V) `dp[i] = dp[i - w]`
- (W) `dp[i] = min(dp[i - 1] + 1, dp[i - w])`

(b) Choose the correct invariant for the for loop in line 5 from the options below:

A. For $j < i$: $dp[j]$ is the minimum number of jumps to reach field j, or it is n if the field is unreachable.

B. For $j < i$: $dp[j]$ is the minimum number of jumps to reach field j, or it is -1 if the field is unreachable.

C. For $j < i$: $dp[j]$ is the minimum number of moves (walks and jumps) to reach field j, or it is n if the field is unreachable.

D. For $j < i$: $dp[j]$ is the minimum number of moves (walks and jumps) to reach field j, or it is -1 if the field is unreachable.

Task 13:

Consider the following decision problem A:

Input: a graph G and two vertices s and t. Is there a path from s to t?

Consider the following decision problem B:

Input: a weighted graph G. Is there a tour going through all the n vertices (i.e., visiting each vertex at least once), such that the total edge weight of the tour is exactly n?

Mark all the correct sentences and give a short explanation.

- √ **A can be reduced to B in polynomial time.**
- ◯ B can be reduced to A in polynomial time.
- √ **A is in P.**
- ◯ B is in P.
- ◯ It is unknown whether A can be reduced to B in polynomial time.
- √ **It is unknown whether B can be reduced to A in polynomial time.**
- ◯ It is unknown whether A is in P.
- √ **It is unknown whether B is in P.**

Solution: Problem B is in NP as we can reduce it to the NP-complete problem Traveling Salesman: Our problem is the special case of TSP where the cost threshold is n. So the reduction is to set the cost threshold C to n.

Note that this reduction shows only that problem B is in NP. It does not show that it is NP-complete.

Problem B is also NP-hard as we can reduce the NP-complete problem Hamiltonian Cycle to it: As shown in the lecture: a graph has a Hamiltonian cycle if and only if it has a cycle visiting each vertex exactly once which is equivalent to the existence of a cycle that uses n edges and visits all the vertices. If we set the edge weights to 1 (this is our reduction), the Hamiltonian Cycle problem is equivalent to the existence of a cycle that has total weight n and that visits all vertices; which is the problem from the task statement..

Since problem B is both, in NP and NP-hard, it is NP-complete.

Problem A can be solved in polynomial time: The problem can be solved in polynomial time using, for instance, breadth-first search.

Consequently, as problem A can be solved in polynomial time, it can reduced in polynomial time to any other problem, thus also to B.

Since it is unknown whether NP-complete problems can be solved in polynomial time, we do not know whether B is in P, and, consequently, whether there exist a polynomial time reduction from B to any polynomial-time problem (e.g., to A).